

软件安全综合实践

实验三：格式化字符串漏洞

学院名称： 智能与计算学部

专 业： 网络空间安全

学生姓名： b1qx

年 班 级： 2024 级

日 期： 2026 年 6 月 25 日

1. 实验目的

- ◆ 了解格式化字符串及 `printf` 函数族的基本工作方式。
- ◆ 理解用户输入直接作为格式化字符串时产生漏洞的原因。
- ◆ 通过 `%x`、`%d` 等格式说明符观察程序对栈中数据的非预期读取。
- ◆ 掌握使用固定格式字符串修复漏洞的方法，形成安全编程意识。

2. 实验环境

操作系统：Windows XP 虚拟机；开发工具：Microsoft Visual C++ 6.0；工程类型：Win32 Console Application；编译配置：Win32 Debug。

3. 实验原理

`printf` 函数的第一个参数是格式化字符串。函数会逐个解析其中的普通字符和格式说明符，例如 `%d`、`%x`、`%s`、`%p` 等。每遇到一个格式说明符，`printf` 就会从调用现场继续取得一个与该说明符相匹配的参数，并按照指定格式进行输出。

正常写法是 `printf("%s", str)`，其中格式字符串固定为“`%s`”，用户输入 `str` 只是普通数据。实验程序使用 `printf(str)`，导致用户输入同时控制了格式字符串。当输入中包含多个 `%x` 或 `%d` 时，`printf` 会误以为调用者还传入了多个整数参数。实际上这些参数并不存在，于是函数会继续读取栈中的其他内容并输出，从而造成内存信息泄露。

在 VC++ 6.0 的 32 位 Debug 环境中，未初始化的栈空间经常被填充为 `0xCCCCCCCC`，因此实验输出中可能出现大量“`CCCCCCCC`”。这属于调试运行库的典型填充值，也进一步说明程序读取了本不应该输出的栈内存。若使用 `%s`，程序会把读取到的数值当作字符串地址并进行解引用，可能造成越界读取或程序崩溃；`%n` 还可能被利用为内存写入。因此本实验只使用 `%x`、`%d` 等格式符进行观察。

4. 过程记录

4.1 创建工程并进行正常输入测试

在 VC++ 6.0 中创建 Win32 Console Application 空工程 `exp3`，新建源文件 `exp3.c`，输入以下代码并在 Debug 模式下编译运行。

```
#include <stdio.h>
```

```

int main()
{
    char str[200];
    fgets(str, 200, stdin);
    printf(str);
    return 0;
}

```

首先输入普通字符串 `hello`。程序能够正常读取并输出相同内容，说明输入、编译和运行环境均正常。此时输入中不包含格式说明符，因此尚未触发漏洞。

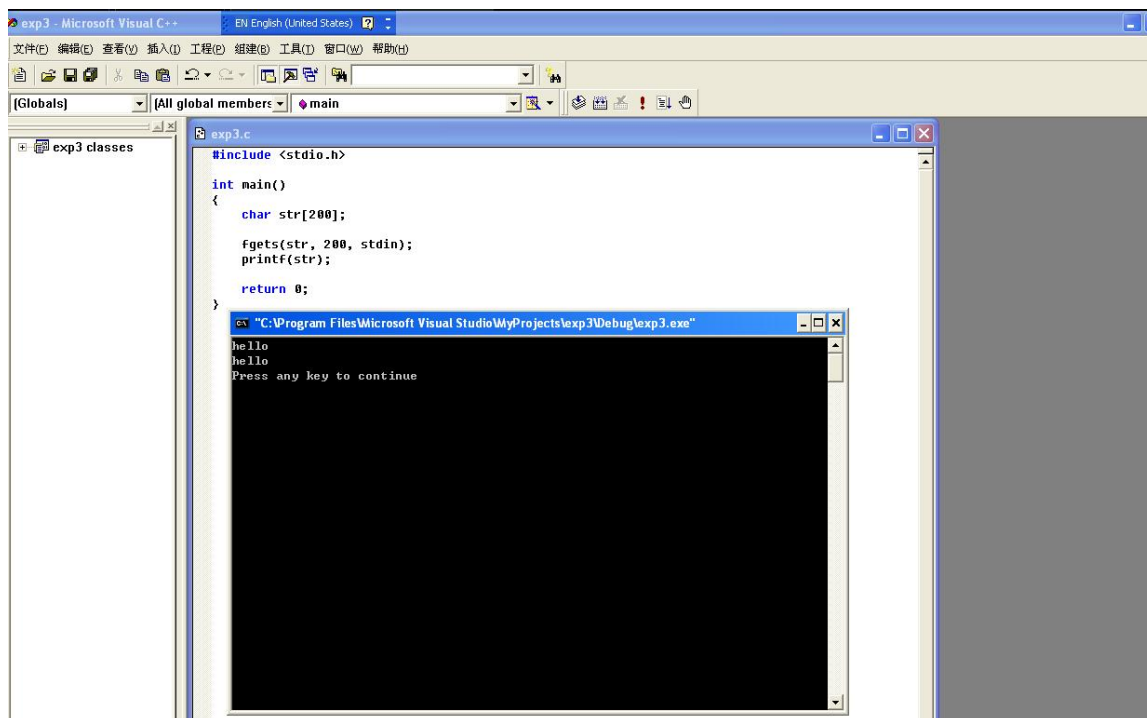


图 1 程序源代码及普通字符串输入测试

4.2 使用 `%x` 观察栈中数据

重新运行程序，输入由“AAAA”和多个“`%x`”组成的字符串。输出结果不再只是原始输入，而是在 AAAA 后出现了多组十六进制数值。`%x` 要求 `printf` 按无符号十六进制整数输出参数，但程序没有提供对应的参数，因此 `printf` 从栈中继续取值并将其输出。

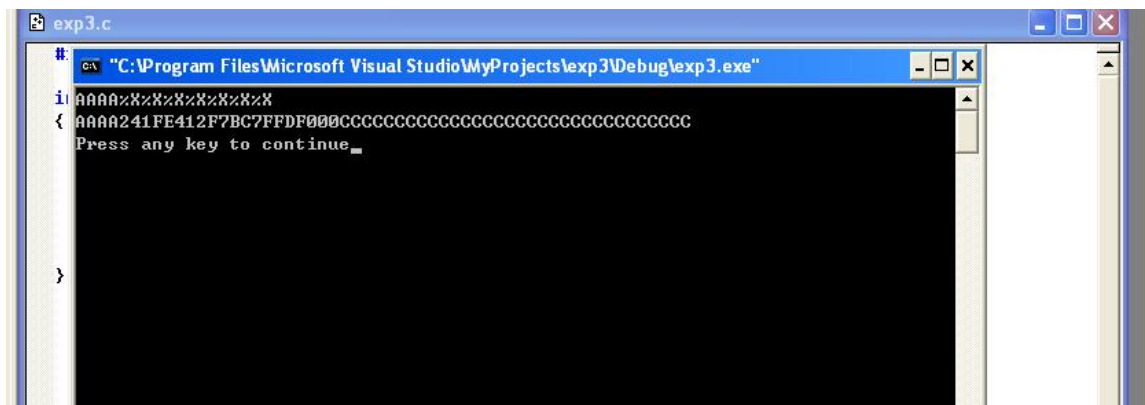


图 2 连续使用 %x 后输出栈中的十六进制数据

截图中出现的 0xCFFFFFFF 一类数据与 VC++ Debug 模式下的栈填充值有关；其他十六进制数可能来自函数调用过程中的地址、临时变量或保存的数据。具体数值会受到运行环境和每次调用时栈状态的影响，因此不同运行结果不必完全一致。

4.3 使用 %d 进行对比测试

随后将格式说明符改为连续多个 %d。%d 同样会继续从栈中读取数据，但会把相同的二进制内容解释为有符号十进制整数，因此输出中既有正数，也可能出现负数。例如 0xCFFFFFFF 按 32 位有符号整数解释时会得到负值。该现象说明格式说明符不仅决定是否读取参数，也决定读取后如何解释和显示数据。

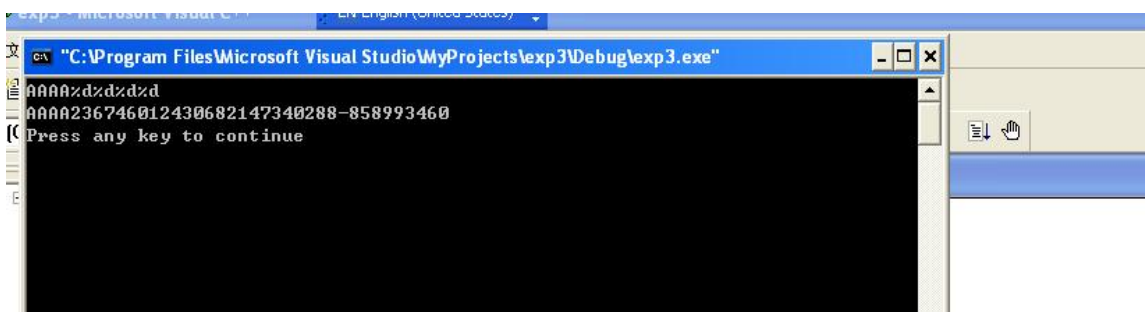


图 3 连续使用 %d 后按有符号十进制输出栈数据

4.4 修改代码并验证修复结果

将存在漏洞的 printf(str) 修改为 printf("%s", str)。修改后，格式字符串由程序固定，用户输入只作为 %s 对应的字符串参数。再次输入包含多个 %x 的内容时，程序会将这些字符原样输出，不再把它们解释为格式说明符。

```
printf("%s", str);
```

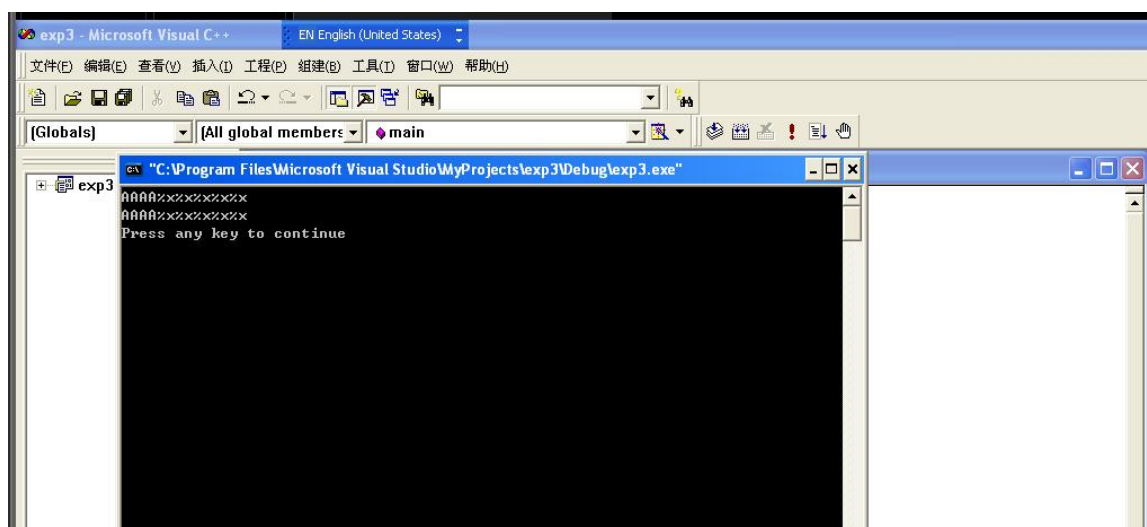


图 4 修复后包含 %x 的输入被原样输出

5. 实验结果与分析

实验成功复现了格式化字符串漏洞。正常输入时程序表现正常；输入多个 %x 或 %d 后，程序读取并输出了调用栈中的非预期数据，证明 `printf(str)` 允许外部输入控制格式解析过程。使用 %x 时数据以十六进制显示，便于观察内存字节模式；使用 %d 时同一数据被解释为有符号十进制，因此可能出现负数。

该漏洞的根本原因不是 `fgets`，而是输出函数的使用方式错误。`fgets` 对输入长度进行了限制，能够避免本例中的缓冲区越界，但无法阻止格式化字符串漏洞。只要不可信输入被直接放在 `printf` 的第一个参数位置，攻击者仍可以构造格式说明符读取内存，严重时还可能修改内存。

修复方法是始终使用固定的格式字符串，例如 `printf("%s", str)`，或者直接使用 `fputs(str, stdout)`。同时应开启编译器警告，检查所有 `printf`、`fprintf`、`sprintf`、`snprintf` 等函数的格式参数，禁止把外部输入直接作为格式化字符串。

6. 心得体会

通过本次实验，我认识到格式化字符串不仅决定输出样式，还会控制可变参数的读取过程。虽然程序只有一行 `printf(str)` 写法不规范，但它会使用户输入参与函数对栈的解析，造成内存信息泄露。实验中不同格式说明符得到不同输出，也帮助我理解了同一段二进制数据可以按照十六进制、有符号整数或地址等不同方式解释。